BUAA-OO-LAB-0 实验报告

思考题

Thinking 0.1

> 执行命令 cat Untracked.txt 和 cat Stage.txt，对比两次运行的结果，体会 README.txt 两次所处位置的不同。

README.txt 在 add 前属于**未跟踪的文件**

README.txt 在 add 后属于**要提交的变更**

> 执行命令 cat Modified.txt，观察其结果和第一次执行 add 命令之前的 status 是否一样，并思考原因

README.txt 属于**尚未暂存以备提交的变更**

不同的原因：add + commit 后该文件进入追踪列表，故修改后的 status 不同

Thinking 0.2

> 仔细看看0.10，思考一下箭头中的 add the file 、stage the file 和 commit 分别对应的是 Git 里的哪些命令呢？

- Add the file:
  - `git add file`
- Stage the file:
  - `git add file`
- Commit:
  - `git commit file -m message`

Thinking 0.3

> 1. 代码文件 print.c 被错误删除时，应当使用什么命令将其恢复？
> 2. 代码文件 print.c 被错误删除后，执行了 `git rm print.c` 命令，此时应当 使用什么命令将其恢复？
> 3. 无关文件 hello.txt 已经被添加到暂存区时，如何在不删除此文件的前提下 将其移出暂存区？

1. `git checkout -- print.c`
2. 如下：
   1. `git reset HEAD print.c` 取消暂存
   2. `git checkout -- print.c` 从暂存区恢复文件到工作区，在没有对应分支时可不加横杠
3. `git rm --cache hello.txt`

Thinking 0.4

》 进行版本回退。执行命令 git reset --hard HEAD^后，再执行 git log，观 察其变化。

commit 为 3 的提交消失

> 找到提交说明为 1 的哈希值，执行命令 git reset --hard 后，再执 行 git log，观察其变化。

commit 为 2, 3 的提交均消失

> 现在已经回到了旧版本，为了再次回到新版本，执行 git reset --hard ，再执行 git log，观察其变化。

回到 commit 为 3 的状态

Thinking 0.5

```
echo first
echo second > output.txt
echo third > output.txt
echo forth >> output.txt
```

output.txt:

```
thirdforth
```

Thinking 0.6

```
# command
#!/bin/bash
echo echo Shell Start...
echo echo set a = 1
echo a=1
echo echo set b = 2
echo b=2
echo echo set c = a+b
echo c=\$[\$a+\$b]
echo echo c = \$c
echo echo save c to ./file1
echo echo \$c\>file1
echo echo save b to ./file2
echo echo \$b\>file2
echo echo save a to ./file3
echo echo \$a\>file3
echo echo save file1 file2 file3 to file4
echo cat file1\>file4
echo cat file2\>\>file4
echo cat file3\>\>file4
echo echo save file4 to ./result
echo cat file4\>\>result
```

```
# test
echo Shell Start...
echo set a = 1
a=1
echo set b = 2
b=2
echo set c = a+b
c=$[$a+$b]
echo c = $c
echo save c to ./file1
echo $c>file1
echo save b to ./file2
echo $b>file2
echo save a to ./file3
echo $a>file3
echo save file1 file2 file3 to file4
cat file1>file4
cat file2>>file4
cat file3>>file4
echo save file4 to ./result
cat file4>>result


# result
3
2
1
```

result 保存的分别是 file1 中 c 的值， file2 中 b 的值和file3 中 a 的值。 更前一步则是来自 test 中对 a,b,c 的赋值，计算和储存。

echo 命令在不同情况下的行为：

- *命令 echo message 中，将输出 message 转义后的内容（似乎有多次转义导致 '\n' 在首次没有作为换行符转义，而是返回 'n'）。
- 命令 echo 'message' 中，将输出 message 的原始内容。
- 命令 echo "message" 中，将输出 message 转义时会无视文件通配符，但会转义 $,\。
- 命令 echo `message` 中，将输出 message 作为命令的执行结果。

所以显然 echo echo Shell Start 和 echo `echo Shell Start` 以及 echo echo $c>file1 与 echo `echo $c>file1` 效果不同。

难点

暂无

实验体会

lab0 的整体难度不大，但是需要学习的新知识很多很杂。

在完成作业的过程中，学习使用了许多的小且有效的工具。

在自行编写库头文件和使用各种脚本把整个项目连接起来之后， 对编译和底层的自动化逻辑有了更深一步的了解， 各类技巧的综合运用，能够使得简单的程序拥有巨大的灵活性。

总而言之，Pre 和 Lab0 给 OS 的学习开了个好头。